

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Proyecto Fase 3:

Quetxal TV

PONDERACIÓN: 20 pts

Índice

Descripción del problema	3
Alcance del sistema	4
1. Motor inteligente de recomendación	4
2. Control Parental (Disponible en todos los planes)	4
3. Watch Party (Exclusivo plan premium)	4
4. Descarga de contenido (Plan Estándar y Premium)	4
5. Tareas Programadas de Mantenimiento (Cronjob de Depuración)	4
Requisitos y restricciones	5
• Validación de reglas de negocio en gRPC:	5
• Persistencia Aislada de Kubernetes:	5
• Infraestructura como código (IaC-Terraform):	5
• Gestión de configuración Automatizada (Ansible):	5
• Punto de entrada ingress:	5
Testing, Calidad y Observabilidad continua	6
• Pruebas Unitarias y de Integración (CI/CD):	6
• Smoke Tests (CI/CD):	6
• Pruebas de carga ligera (Locust) (Nube):	6
• Stack de Observabilidad de Logs (ELK Stack) (Nube):	6
• Stack de Observabilidad de Métricas (Prometheus y Grafana) (Nube):	6
Presentación y Defensa del proyecto	6
• Estructura Obligatoria de la presentación:	6
Entregables Requeridos	7
• Modelado y justificación Arquitectónica	7
• Manuales de infraestructura, operaciones y pruebas:	7
• Actualización general de Diagramas	8
1. Entregables de Integración y Pruebas:	8
2. Archivos de configuración:	8
Herramientas permitidas	9
Cronograma	10
Consideraciones	10

Descripción del problema

Con una arquitectura polígloa, un pipeline automatizado de CI/CD y un catálogo administrado en producción, el reto final se enfoca en dotar a la plataforma de capacidades inteligentes para la retención de usuarios y asegurar que toda la infraestructura sea replicable, monitoreable de manera proactiva y tolerante a ráfagas de concurrencia masiva.

En el mercado global, la infraestructura no se aprovisiona de forma manual ni se configura entrando por SSH a cada servidor; se codifica y destruye bajo demanda. Asimismo, el departamento de operaciones necesita visibilidad total sobre la salud de los contenedores y los cuellos de botella mediante la correlación de logs y métricas antes de que afecten la experiencia del cliente. Finalmente, el motor de base de datos relacional debe ser aislado completamente del ciclo de vida efímero de los contenedores de aplicación, garantizando la persistencia e integridad de datos críticos fuera del clúster de cómputo.

Alcance del sistema

La aplicación web debe resolver de extremo a extremo los siguientes módulos y capacidades de negocio:

1. Motor inteligente de recomendación

- Implementación obligatoria en el backend de uno de los algoritmos de recomendación icónicos de Netflix (ej. *Filtrado Colaborativo basado en usuarios/items* o *Sistemas de Recomendación Basados en Contenido/Géneros*). .
- El sistema debe analizar el historial de reproducción reciente de un perfil y sus calificaciones previas para generar dinámicamente en el frontend una sección personalizada titulada "**Recomendados para ti**".

2. Control Parental (Disponible en todos los planes)

- Clasificación estricta del contenido multimedia en el catálogo (ej. Apta para todo público, PG-13, R).
- Permitir que la cuenta configure un PIN de seguridad restrictivo de 4 dígitos para perfiles específicos (perfiles infantiles). El sistema bloqueará la reproducción de contenidos no aptos a menos que se introduzca el PIN correcto.

3. Watch Party (Exclusivo plan premium)

- Creación de salas de reproducción sincronizada en tiempo real utilizando WebSockets.
- **Restricción de Plan:** Solo los usuarios con suscripción de **Plan Premium** tienen el privilegio de *iniciar y crear* una Watch Party. Sin embargo, una vez generada la sala, el usuario premium **puede invitar a cualquier tipo de usuario** (Básico o Estándar) mediante un enlace/código para unirse a la sesión sincronizada.

4. Descarga de contenido (Plan Estándar y Premium)

- Módulo para habilitar la descarga simulada o almacenamiento local de contenidos de video (utilizando almacenamiento cifrado del navegador o Service Workers).
- **Restricción de Plan:** Esta funcionalidad debe estar bloqueada para el Plan Básico y el Plan Premium (la descarga queda segmentada normativamente **únicamente para el Plan Estándar**).

5. Tareas Programadas de Mantenimiento (Cronjob de Depuración)

- Creación de un proceso de fondo configurado como una tarea cronometrada (**Cronjob**) encargada de auditar la base de datos de manera automatizada.
- El proceso debe buscar y eliminar lógicamente (o purgar) aquellas cuentas de usuarios que permanezcan inactivas o sin registros de inicio de sesión

durante un periodo de tiempo prefijado de X tiempo (Coloquen un tiempo prudencial para la calificación).

Requisitos y restricciones

El Pipeline de CI/CD debe configurarse bajo la premisa de **cortocircuito crítico**: **si ocurre un fallo en las pruebas, la compilación o los scripts, el pipeline detendrá su ejecución de inmediato** impidiendo que el código progrese a etapas de empaquetado o despliegue.

- **Validación de reglas de negocio en gRPC:**

Los microservicios de streaming y salas de Watch Party deben implementar *interceptores gRPC* para validar, antes de procesar la petición, que el token JWT del usuario posea el rol y el tipo de plan adecuado (Estándar para descargas, Premium para iniciar Watch Party) y que se cumplan las políticas de Control Parental.

- **Persistencia Aislada de Kubernetes:**

Queda estrictamente prohibido desplegar los motores de base de datos relacionales u operacionales dentro de los Pods del clúster de Kubernetes. Las bases de datos deben residir en servidores externos (instancias dedicadas de Compute Engine).

- **Infraestructura como código (IaC-Terraform):**

La creación, modificación y destrucción de toda la infraestructura (VPC, subredes, firewalls, clúster de GKE, VMs de desarrollo y servidores de BD externos) debe ser declarativa mediante archivos de **Terraform**.

- **Gestión de configuración Automatizada (Ansible):**

El despliegue de dependencias, herramientas base y preparación de los entornos de ejecución en las VMs se automatizará exclusivamente por medio de Playbooks de **Ansible**.

- **Punto de entrada ingress:**

Se mantiene la obligatoriedad de utilizar un recurso **Ingress** en GKE como el único interceptor de tráfico web externo, gestionando las reglas de enrutamiento hacia el API Gateway.

Testing, Calidad y Observabilidad continua

- **Pruebas Unitarias y de Integración (CI/CD):**
Verificación automática del backend manteniendo el umbral mínimo estricto del **75% de cobertura de endpoints**.
- **Smoke Tests (CI/CD):**
Tras realizar el despliegue automático en el entorno correspondiente, el pipeline gatillará suites de *Smoke Tests* automáticos para certificar que las rutas base del API Gateway y las conexiones críticas están en línea.
- **Pruebas de carga ligera (Locust) (Nube):**
Implementación de scripts de simulación de usuarios concurrentes utilizando **Locust**. Se simulará estrés inyectando tráfico masivo en las rutas críticas, Al finalizar las pruebas se requiere generar un archivo HTML con los resultados de las pruebas.
- **Stack de Observabilidad de Logs (ELK Stack) (Nube):**
Configuración centralizada de **Elasticsearch, Logstash y Kibana** para recolectar y centralizar los logs de auditoría de todos los contenedores y servidores externos.
- **Stack de Observabilidad de Métricas (Prometheus y Grafana) (Nube):**
Recolección de métricas de hardware y de red en tiempo real a través de **Prometheus** y visualización centralizada en dashboards interactivos en **Grafana**.

Presentación y Defensa del proyecto

- **Estructura Obligatoria de la presentación:**
 - **El Problema Inicial:** Diagnóstico de la situación del negocio, cuellos de botella identificados en plataformas monolíticas y justificación de la necesidad del cambio.
 - **Planteamiento de la Solución (Toma de Decisiones):** Explicación analítica y técnica de las decisiones arquitectónicas adoptadas. Se debe fundamentar qué análisis técnicos y comparativas del mercado se realizaron para seleccionar la matriz políglota, las bases de datos externas, los esquemas de sesión, las herramientas IaC (Terraform/Ansible) y los stacks de observabilidad.
 - **La Solución Final:** Demostración del ecosistema operativo unificado, evidenciando cómo la infraestructura elástica y el backend inteligente resuelven las necesidades del negocio bajo altos estándares de disponibilidad.

- Restricciones de Tiempo y Participación:
 - **Límite Máximo de Tiempo:** La presentación tendrá una duración máxima estricta de **20 minutos**.
 - **Participación de Integrantes:** **Todos** los integrantes del grupo de desarrollo deben exponer y hablar activamente.
 - **Atributos de Calidad:** Cada estudiante debe expresarse de forma técnicamente correcta, expresiva y fluida, utilizando el vocabulario ingenieril adecuado para la defensa del proyecto.

Entregables Requeridos

- **Modelado y justificación Arquitectónica**

Documentación del Algoritmo de Recomendación: Explicación teórica del algoritmo de Netflix seleccionado, diseño del modelo matemático/lógico y su implementación.

- **Manuales de infraestructura, operaciones y pruebas:**

- **Documentación sobre terraform:**
 - **Qué es y Cómo funciona:** Marco teórico del aprovisionamiento declarativo y la gestión del archivo de estado.
 - **Configuración paso a paso de la infraestructura:** Guía detallada que documente la creación de la VPC, subredes, firewalls, clúster de GKE, instancias de base de datos externas y VMs. **Es obligatorio incluir capturas de pantalla** que evidencien los recursos levantados mediante el código IaC.
- **Documentación sobre Ansible:**
 - **Qué es y Cómo funciona:** Marco teórico de la automatización sin agentes (*agentless*) mediante conexiones SSH y la estructura de Playbooks/Roles.
 - **Configuración paso a paso:** Detalle de cómo se aprovisionan las herramientas base y entornos en las VMs de desarrollo. **Se deben incluir capturas de pantalla** de los logs de ejecución de los playbooks en la terminal.
- **Documentación sobre el Stack ELK:**
 - **Qué es y Cómo funciona:** Explicación de la arquitectura de recolección de logs (Elasticsearch para almacenamiento, Logstash para filtrado/transformación y Kibana para visualización).
 - **Configuración paso a paso:** Flujo detallado de la inyección de agentes o redirección de streams de salida de los microservicios. **Es obligatorio incluir capturas de pantalla de Kibana** mostrando la indexación de los logs transaccionales y de auditoría del sistema.

- **Documentación sobre Prometheus y Grafana:**
 - **Qué es y Cómo funciona:** Explicación del modelo de monitoreo basado en series temporales por recolección activa (*scraping*) y el aprovisionamiento de métricas en tableros.
 - **Configuración paso a paso:** Guía del despliegue de los exporters en el clúster y servidores externos. **Es obligatorio incluir capturas de pantalla de los Dashboards de Grafana** reflejando la telemetría viva del sistema.
- **Documentación de Locust:**
 - **Qué es y Cómo funciona:** Teoría de las pruebas de carga distribuida basadas en código de Python que simulan flujos de usuarios reales.
 - **Resultados de las pruebas con capturas:** Documentación de los escenarios de estrés ejecutados. **Es obligatorio incluir las capturas de los resultados de Locust.**
- **Actualización general de Diagramas**
 - Adaptación completa de requerimientos ampliados, diagramas de casos de uso del administrador (con narrativas expandidas y flujos de excepción técnicos) y actualización del Modelo de 4+1 Vistas de Kruchten para acoplar la infraestructura actual.
 - Diagrama de Arquitectura de alto nivel incluyendo las herramientas de monitoreo
 - Diagrama de flujo de CI/CD con las nuevas herramientas de testing
 - Documento de justificación de herramientas utilizadas
- 1. **Entregables de Integración y Pruebas:**
 - Historial de Git e Integración (Pull Requests): Evidencia en el repositorio del uso correcto de ramas y los Pull Request aprobados para la consolidación del proyecto.
 - Creación del Tag con la Versión V2.0.0.
- 2. **Archivos de configuración:**
 - Se deben agregar de forma obligatoria los archivos Dockerfile por servicio y los dos archivos Docker Compose utilizados para desplegar en el entorno local y en la nube.
 - Manifiestos y Configuraciones de Objetos de Kubernetes
 - Archivos de configuración para pipeline CI/CD y scripts (Si es que se utilizaron).

Herramientas permitidas

Tipo	Categoría	Descripción
Obligatorio	Lenguajes	Go, TypeScript, Python
Opcional	Framework	FastAPI, Flask, Python, Express, NestJS, Gin
Obligatorio	Comunicación	gRPC
Obligatorio	Contratos	Protocol Buffers
Opcional	Seguridad y Sesiones	JWT, Session Cookies, OAuth
Obligatorio	Almacenamiento Caché	Redis
Opcional	Base de datos	MSSQL, MySQL, PostgreSQL, MongoDB
Obligatorio	Contenedores	Docker
Obligatorio	Orquestación	Docker-Compose, Kubernetes
Obligatorio	Control de Versiones	Github
Obligatorio	Automatización CI/CD	GithubActions
Obligatorio	Nube	Google Cloud Platform
Obligatorio	Almacenamiento de objetos	Google Cloud Storage (GCS)
Obligatorio	Despliegue	Google Compute Engine, Google Kubernetes Engine
Obligatorio	IaC	Terraform, Ansible
Opcional	Documentación, Diseño, Modelado	Excalidraw, Figma, Canva, LucidChart, Draw.io , StarUML, Visual Paradigm, FossFlow

Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	20/06/2026	29/06/2026
Elaboración	20/06/2026	29/06/2026
Calificación	29/06/2026	30/06/2026

Consideraciones

- **Fecha límite de entrega:** 29 de Junio a las 09:00 AM
- **Nombre del repositorio:** SA_PROYECTO_GX
- **Colaborador:** *Samashoas*
- **Medio de entrega:** UEDI
- **Documento Técnico:** Formato Markdown que incluya tabla de integrantes, índice, introducción, desarrollo de todos los diagramas y conclusiones.
- No se permite el uso de herramientas como supabase o prisma para la gestión de la base de datos
- No se calificará ninguna funcionalidad en el entorno local, solo se calificarán funcionalidades en el entorno de la nube.
- Se deben cargar los archivos crudos de la documentación al repositorio, de lo contrario el diagrama no será válido.
- Se calificará en base a la documentación, si algún elemento no se encuentra documentado no será tomado en cuenta para la calificación.
- No se permite realizar despliegues, configuraciones, cargas de datos y accionamiento del CI/CD POR NINGÚN MOTIVO.
- Los sistemas como GKE y Compute Engine ÚNICAMENTE SE PUEDEN DAR DE BAJA MOMENTÁNEA SOLAMENTE DESPUÉS DE LA CALIFICACIÓN.